

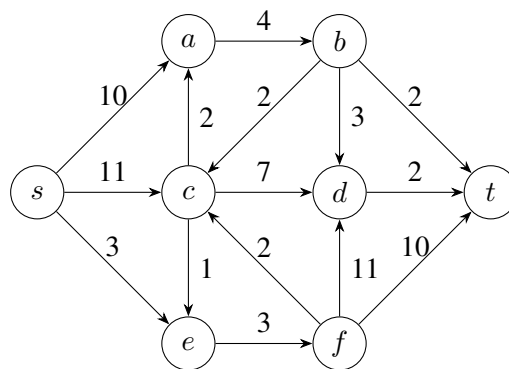
Problem Set 2

CMSC 27200: Theory of Algorithms

Assigned January 14, 2026, Due January 20, 2026

Problem 1: Dijkstra's Algorithm (25 points)

Use Dijkstra's algorithm to find the shortest path(s) from s to t in the following graph G .



For your answer, first give the state of the priority queue after we add in the edges going out from s . Then give the following data for each new vertex that is reached.

1. The name of the vertex v which is reached and the length of the shortest path from s to v .
2. The state of the priority queue after we add in the edges which go out from v .

After giving this data, find **all of the shortest paths** from s to t .

Note: For this problem, you do not need to show the heap structure of the priority queue. Instead, you can represent the priority queue as a sorted list. Also, you may use either a priority queue with edges or a priority queue with vertices (where the value for a vertex decreases when a shorter path to that vertex is found).

Problem 1 Solution

Applying Dijkstra's algorithm with a priority queue containing vertices and reducing the distance to a vertex when a shorter path is found:

Vertex name	Vertex distance	Priority queue state
s	0	$(e, 3), (c, 11), (a, 10)$
e	3	$(f, 6), (c, 11), (a, 10)$
f	6	$(c, 8), (a, 10), (d, 17), (t, 16)$
c	8	$(a, 10), (d, 15), (t, 16)$
a	10	$(b, 14), (d, 15), (t, 16)$
b	14	$(d, 15), (t, 16)$
d	15	$(t, 16)$
t	16	

Applying Dijkstra's algorithm with a priority queue containing edges:

Vertex name	Vertex distance	Priority queue state
s	0	$(s-e, 3), (s-c, 11), (s-a, 10)$
e	3	$(e-f, 6), (s-c, 11), (s-a, 10)$
f	6	$(f-c, 10), (s-c, 11), (s-a, 10), (f-d, 17), (f-t, 16)$
c	8	$(c-d, 15), (c-a, 10), (s-a, 10), (f-d, 17), (f-t, 16)$
a	10	$(a-b, 14), (f-d, 19), (c-d, 19), (f-t, 20)$
b	14	$(f-d, 19), (c-d, 19), (f-t, 20), (b-c, 16), (b-t, 16), (b-d, 17)$
d	15	$(f-t, 20), (b-c, 16), (b-t, 16), (b-d, 17), (d-t, 16)$
t	16	$(b-c, 16), (b-t, 20), (b-d, 17), (d-t, 16)$

There are two shortest paths of length 16: $s-e-f-t$ and $s-e-f-c-a-b-t$.

Problem 2: Guarding the Vault (25 points)

You are in charge of guarding a vault. You would like to take a vacation from time 0 to time T , but first you need to hire other guards to guard the vault while you are away (it is okay if more than one person is guarding the vault, but the vault must be guarded at all times). You know n guards you could call, each of whom has a set time interval $[a_i, b_i]$ when they can come and guard the vault. For all $t \in [0, T]$, there is at least one i such that $t \in [a_i, b_i]$. Each guard charges the same amount regardless of the length of their interval $[a_i, b_i]$, so you would like to hire as few guards as possible.

Give a polynomial time greedy algorithm to minimize the number of guards that you need to hire, prove that your algorithm is correct, and analyze its runtime.

Note: We can state this problem mathematically as follows. Find a set $I \subseteq [n]$ such that $[0, T] \subseteq \cup_{i \in I} [a_i, b_i]$ and $|I|$ is minimized.

Problem 2 Solution

Without loss of generality, we can perturb the intervals (by replacing $[a_i, b_i]$ with $[a_i - \epsilon, b_i + \epsilon]$) so that all intervals endpoints are distinct without changing the validity of any schedules.

We pick intervals $[a_{i_1}, b_{i_1}], [a_{i_2}, b_{i_2}], \dots, [a_{i_k}, b_{i_k}]$ one at a time using a greedy strategy: if the last interval we chose was $I_{i_j} := [a_{i_j}, b_{i_j}]$, we choose the interval $I_{i_{j+1}}$ to be the interval containing b_{i_j} with the latest end time (or at the very first step, the interval containing 0 with the latest end time).

We can prove the optimality of this algorithm by contradiction. Let S be the schedule produced by our greedy algorithm and S' any other schedule. Let $f(S, t)$ (resp. $f(S', t)$) be the number of intervals in S (resp. S') which end at or before t . Let b be the smallest value such that $f(S, b) > f(S', b)$. By minimality of b , there is an interval $I_j := [a_{i_j}, b_{i_j}] =: [a, b]$ in S . Because $f(S, t) = f(S', t)$ for all $t < b$, the set of intervals in S and S' ending strictly before b must be the same. Sorting the intervals in S by their end time, let $I_{j-1} := [a_{i_{j-1}}, b_{i_{j-1}}]$ be the interval in S immediately preceding $[a, b]$. Because S' is also a schedule, there is an interval $[a', b']$ in S' containing $b_{i_{j-1}}$, but because $f(S, b) > f(S', b)$, we must have $a' < b_{i_{j-1}} < b < b'$. This is a contradiction, because after choosing I_{j-1} , the greedy algorithm would have chosen $[a', b']$ over $[a, b]$ since both contain $b_{i_{j-1}}$ but $b' > b$.

We can implement this algorithm by sorting the list of intervals by their ending time and, after choosing $I_j := [a_{i_j}, b_{i_j}]$, picking I_{j+1} to be the interval with the latest end time containing b_{i_j} . The total number of intervals we choose in this way is at most $O(n)$ and we take $O(n)$ time to find I_{j+1} given I_j since we may need to search through the whole list, so this gives an algorithm running in $O(n^2)$ time.

We can improve this to $O(n \log n)$ if we can (after at most $O(n \log n)$ precomputation), find the latest-ending interval containing b_{i_j} in $O(\log n)$ time. Data structures designed for such queries are interval and range trees – variants of binary search trees – and either would work here. Here is a more down-to-earth approach. Let $(t_j)_{j=1}^{2n}$ be a list containing all interval start and end times. It suffices to be able to compute, for each t_j , the intervals containing t_j sorted by ending time. For this, let L_{j-1} be a binary search tree containing the intervals containing t_{j-1} ordered by ending time. If t_j is an interval start (resp. end) point, we insert (resp. remove) interval I_j into (resp. from) L_{j-1} to obtain L_j . Both insertion and deletion from a binary search tree take $O(\log n)$ time, and we process each interval exactly twice, so computing the BSTs $\{L_j\}_{j=1}^{2n}$ takes $O(n \log n)$ time. With these precomputations in hand, we can execute the greedy algorithm in $O(n \log n)$ time.

We cannot hope to execute this greedy algorithm in better than $O(n \log n)$ time since already at the first step we need to sort the intervals by end time.

Problem 3: Maximum Cardinality Matching in a Rooted Tree (20 points, 5 points extra credit available)

Given an (undirected) graph $G = (V, E)$, a matching S is a set of edges such that no two edges of S have a common endpoint. A maximum cardinality matching is a matching M with the maximum number of edges in it (i.e., for all matchings S , one has $|S| \leq |M|$).

- (a) 10 points: Give a polynomial time greedy algorithm that finds a maximum cardinality matching in an (undirected) rooted tree $T = (V, E)$. You need to show that your algorithm runs in polynomial time and justify its correctness.

Note: In this part of the problem, you are allowed to use “abstract” operations, such as “removing an edge from the graph”, “finding a node in the graph that satisfies some required

properties”, without having to specify how to implement such operations (in fact, the implementations depend on how the tree T is represented, which we deliberately not specify in this part of the problem). However, you should specify what these operations do and argue that they can be implemented in polynomial time.

- (b) 10 points: Suppose the input rooted tree $T = (V, E)$ in part (a) is given by the adjacency lists of its vertices. Specifically, its vertices are $V = \{1, \dots, n\}$, and for each node $v \in V$, its children are given as an array $\text{child}(v)$. Show how to implement your greedy algorithm from part (a). You need to specify the data structure that you use in your implementation, and analyze the runtime of your implementation of the algorithm.
- (c) 5 points extra credit: Give an algorithm (together with its implementation using a data structure) for part (b) whose runtime is optimal, i.e., the best possible up to a constant factor. You should analyze the runtime of your implementation and justify why this is optimal.

Note: you are allowed to use the same algorithm and implementation from part (b), and if you do so, you don’t need to analyze its runtime again.

Problem 3 Solution

We start with a preliminary lemma. Given a rooted tree T , let $f(T)$ denote the maximum cardinality of a matchings in T .

Lemma 0.1. *Let T be a rooted tree with root r and $\{T_i\}$ the set of subtrees rooted at the children of r . Then*

$$\sum_i f(T_i) \leq f(T) \leq 1 + \sum_i f(T_i)$$

Proof. The lower bound is because a matching on all the subtrees separately gives a matching on T . The upper bound is because any matching contains at most one edge containing r , and having chosen such an edge, all the other edges must give a matching on the $\{T_i\}$. $\square$

We abbreviate ”maximal cardinality matching” as MCM. It follows from the lemma that the cardinality of an MCM on T is $1 + \sum_i f(T_i)$ if and only if any MCM must include an edge containing r – if an MCM does not hit r , then the MCM gives an MCM on $\sqcup_i T_i$, and if the cardinality of an MCM on T exceeds that of $\sqcup_i T_i$ then it had better make use of some edges not contained in T , i.e. the edges containing r .

- (a) Our algorithm $\mathcal{A}$ to compute an MCM on T runs as follows. If T is a single vertex, $\mathcal{A}(T)$ returns the empty set. Otherwise, $\mathcal{A}(T)$ returns $\cup_i \mathcal{A}(T_i)$, as well as (r, r_j) where j is the smallest i such that r_i , the root of T_i , is not contained in the matching produced by $\mathcal{A}(T_i)$, if such an i exists. We claim that $\mathcal{A}(T)$ produces an MCM. Furthermore, we claim that this MCM includes an edge containing the root r if and only if the cardinality of the MCM is $1 + \sum_i f(T_i)$. Observe that this happens if and only if any MCM includes a edge containing r (as an MCM avoiding r cannot have size exceeding $\sum_i f(T_i)$). Intuitively, this means that the MCM only includes an edge containing r if it absolutely needs to.

We prove this claim by induction on the height of the tree. The base case of a single vertex is immediate since the MCM is empty. Otherwise, applying the inductive hypothesis to the

subtrees $\{T_i\}$, we can produce an MCM of $\sqcup_i T_i$. We are able to include an additional edge (attaining the upper bound of the lemma) if and only if there is an MCM on some T_i which does not include its root r_i . By the inductive hypothesis, the MCMs produced by $\mathcal{A}(T_i)$ will avoid the root r_i if possible. If we can avoid it for some T_i , then $f(T) = 1 + \sum_i f(T_i)$ and our algorithm produces a matching including the edge (r, r_i) . Otherwise, $\mathcal{A}$ returns $\sqcup \mathcal{A}(T_i)$, which does not include any edge containing r . In either case, we have optimality by the lemma.

We start at each leaf and work our way up. The processing at each vertex involves looking through the children of the vertex and seeing if any of its children subtrees have MCMs that do not include the subtree root.

- (b) We use the given adjacency list representation of the graph, but at each vertex v , we also store the parent of v and an edge from v to one of its children if the MCM on the subtree rooted at v includes such. Traversing the tree "bottom up", starting from the leaves, and applying $\mathcal{A}$, we can update these extra fields as we make our way up the tree as defined in the previous part. Given the data of this boolean for every vertex in the tree, we can reconstruct the matching itself. We process each edge exactly once in this algorithm, so the runtime is $O(m) = O(n)$.
- (c) The algorithm we give is $O(m) = O(n)$. This is optimal as we need at least this long to read the input.

Problem 4: Denominations of Currencies (20 points)

- (a) 10 points: A bank ATM machine has (sufficiently many) bills in the denominations \$1, \$5, \$10, \$20, \$50. Design a polynomial time greedy algorithm that, given a request for providing \$ N of cash, satisfies the request by providing a list of the number of bills of each currency, such that the total number of bills is minimum.

For example, if $N = 87$, the correct answer should be: 1 \$50 bill, 1 \$20 bill, 1 \$10 bill, 1 \$5 bill and 3 \$1 bill. You should prove that your algorithm is correct.

- (b) 5 points: Suppose that in a different country, the available denominations were 1, 5, and 7 units of currency. Show that the greedy algorithm in part (a) is not optimal.
- (c) 5 points: Suppose that the country in part (b) ran out of bills for 1 unit of currency, so the only available denominations are 5 and 7 units of currency. Clearly, with those bills you will not be able to do payments of 1, or 2, or 8. Prove that you will be able to make payments for every sufficiently large natural number n . (More precisely, show that there exists $n_0 > 0$ such that for every $n \geq n_0$, there are natural numbers u, v such that $n = 5u + 7v$).

Problem 4 Solution

- (a) Given a target value N and bill values $\{a_i\}_{i=1}^k$ where $a_1 = 1$ and $a_i < a_{i+1}$, our greedy algorithm decomposes N by choosing $n_k := \left\lfloor \frac{N}{a_k} \right\rfloor$ bills of value a_k , then $\left\lfloor \frac{N - n_k a_k}{a_{k-1}} \right\rfloor$ bills of value

a_{k-1} , and so on. (Because a_1 is 1, this algorithm is guaranteed to output a decomposition of N). The greedy algorithm produces the unique decomposition of N such that the bill with value a_i appears at most $\left\lfloor \frac{a_{i+1}}{a_i} \right\rfloor$ times.

Lemma 0.2. *The greedy algorithm is optimal (produces a decomposition using the fewer number of bills) whenever $a_i | a_{i+1}$ for all $i \leq k - 1$.*

Proof. This follows from an exchange argument. In a decomposition where the i^{th} bill is used at least $\frac{a_{i+1}}{a_i}$ times, we can replace $\frac{a_{i+1}}{a_i}$ of them these bills with one bill of value a_{i+1} . This will improve the decomposition (i.e. reduce the total number of bills). Applying this procedure starting at $i = 1$ and working our way up to $i = k$, we end up with a decomposition where the i^{th} bill is used at most $\frac{a_{i+1}}{a_i}$ times. This is exactly the decomposition produced by the greedy algorithm. $\square$

Thus, the greedy algorithm gives optimal decompositions given the currency system $(1, 5, 10, 20)$, and we need to check that adding 50 does not break optimality. Again we may apply an exchange argument. If a decomposition has three 20s (i.e. if the decomposition has more 20s than in the greedy decomposition), we may improve the decomposition by replacing them with one 50 and one 10. Doing this repeatedly, we end up with the decomposition produced by the greedy algorithm.

Remark 0.3. *Since this is a mistake I (Abhijit) initially made, I want to remark that it is not true that the greedy algorithm works whenever $2a_i \leq a_{i+1}$. For example, consider $(a_i)_i = (1, 2, 4, 14, 35)$. Then, $N = 42 = 35 + 4 + 2 + 1 = 3 \cdot 14$, so the greedy algorithm is not optimal.*

- (b) Observe that $10 = 7 + 1 + 1 + 1 = 5 + 5$. The greedy algorithm would choose the former decomposition, but this is not optimal.
- (c) Observe that if for some m there is a representation $m = 5u + 7v$, then there is such a representation for any $n > m$ for which $n \equiv m \pmod{5}$. Observe that $7 \equiv 2 \pmod{5}$, $14 \equiv 4 \pmod{5}$, $21 \equiv 1 \pmod{5}$, and $28 \equiv 3 \pmod{5}$, so to ensure that we've covered all the residue classes, it suffices to take $(28, 29, 30, 31, 32)$. We can do better by starting with $n_0 = 24$, so that way we cover 28 "just in time".